
Local Machine Setup Guide

What to install, which databases to provision, and how to configure a fresh local machine to run this application end to end.

This guide walks through installing and configuring the Text-to-SQL Dashboard on a machine other than the one it was originally built on — what software to install, which database(s) you need, and exactly what to put in `.env`.

Generated 2026-09-19 by `scripts/build_setup_guide_pdf.py` from `SETUP_GUIDE.md` — the single source of truth for this document. Re-run the script after editing the markdown; do not hand-edit this PDF.

Contents

1. What you need before you start	4
2. Step 1: Get the code and set up Python	5
3. Step 2: Install and start Ollama	6
4. Step 3: The business database	7
5. Step 4: Configure .env for the business database	8
6. Step 5: Verify the connection and build the schema index	9
7. Step 6: Business-context knowledge retrieval (optional)	10
8. Step 7: User accounts and chat history (optional)	11
9. Step 8: Build the frontend	13
10. Step 9: Run the application	14
11. Step 10: Verify it's working	15
12. Full .env reference for a fresh machine	16
13. Alternative: running everything with Docker	18
14. Common problems on a fresh machine	19
15. Security checklist before relying on this outside your own laptop	20

It assumes no prior familiarity with this specific project. If you get stuck, [docs/TROUBLESHOOTING.md](#) and [README.md](#) in the repository root have more detail than this guide repeats; this document is the condensed, setup-focused path from a bare machine to a running app.

There are two databases you may need, and only one of them is required:

- **Your own business database** (PostgreSQL, MySQL, SQL Server, or Oracle) — **required**. This is the database the AI actually turns your questions into SQL against. You already have this; you are not asked to create new data, only to point the app at it with a read-only account.
- **A local identity database** (PostgreSQL only) — **optional**. This is a small, separate database this app creates and manages itself, used only if you want user accounts, login, and chat history that follows a person across browsers and devices. If you skip it, the app still runs with no login screen at all.

Everything else this app depends on (the local LLM via Ollama, the vector index used for schema/knowledge retrieval) runs entirely on the same machine — no other servers and no cloud accounts are required for a basic setup.

1. What you need before you start

- **Python 3.11.** Confirm with `python --version`. If your machine only has a newer Python (3.12+), install 3.11 alongside it rather than replacing your default — see this guide's troubleshooting section if you hit dependency build errors on a newer Python.
- **Node.js + npm** (a recent LTS release, 18 or newer). Only needed to build the React dashboard; skip it if you only intend to call the REST API directly.
- **Git**, to clone the repository.
- **[Ollama](#)**, the local LLM runtime this app uses instead of a cloud AI API. Install it for your operating system from that site.
- **Your business database's connection details:** host, port, database name, and a **read-only** username/password. Supported types: PostgreSQL, MySQL, SQL Server, and Oracle.
- **(Windows + SQL Server only)** the Microsoft ODBC Driver for SQL Server, installed as a system package — this is the one extra manual install step on a fresh Windows machine if your business database is SQL Server. It is not required for PostgreSQL, MySQL, or Oracle.
- **(Optional, for user accounts/chat history)** a local PostgreSQL server, or Docker to run one in a container — covered in Step 7.
- **(Optional)** Docker + Docker Compose, if you'd rather run the whole application in containers instead of following Steps 1–10 by hand — see Step 13.

No sample database is bundled with this project. You must have your own database to connect to before the app is useful.

2. Step 1: Get the code and set up Python

```
git clone https://github.com/surajkumarnavodya/text-to-sql-agent-langgraph.git
cd text-to-sql-agent-langgraph
python -m venv .venv
```

Activate the virtual environment:

```
# Windows (PowerShell)
.venv\Scripts\Activate.ps1

# macOS / Linux
source .venv/bin/activate
```

Then install Python dependencies:

```
pip install --upgrade pip
pip install -r requirements.txt
```

Finally, create your own configuration file from the template — you will edit this throughout the rest of this guide:

```
# Windows
Copy-Item .env.example .env

# macOS / Linux
cp .env.example .env
```

`.env` is listed in `.gitignore` and is never committed — it is safe to put real credentials in it on this machine.

3. Step 2: Install and start Ollama

Ollama runs the AI model locally, so no API key and no network call leaves this machine for question-answering itself. After installing Ollama, pull the default model (or another supported one):

```
ollama pull llama3.1:8b
```

Ollama must be running as a background service before you start this application. On Windows and macOS, the installer typically registers it to run automatically; on Linux, or if it isn't already running, start it with:

```
ollama serve
```

`.env`'s `OLLAMA_HOST` (default `http://localhost:11434`) must point at wherever this service is actually listening — leave it as the default unless you installed Ollama on a different host or port.

4. Step 3: The business database

This is the database the AI reads your schema from and generates SQL against. It is **never modified** by the app — every query is validated as read-only before it runs, and the guidance below adds a second, independent layer of protection at the database itself.

Use a dedicated, read-only database account — not an administrator login. Create a role/user on your database server that:

- Can connect to the target database.
- Has [SELECT](#) privileges on the schema(s) you want the AI to be able to query.
- Has **no** [INSERT/UPDATE/DELETE/DDL](#) privileges at all.

This one step is the most important security decision in this entire setup — the SQL validator described in [SECURITY.md](#) is an additional safeguard, not a substitute for a genuinely read-only database account.

Supported database types and their driver notes:

Database type	Notes
PostgreSQL	No extra driver install needed — psycopg2-binary is already in requirements.txt .
MySQL	No extra driver install needed.
SQL Server	Requires the Microsoft ODBC Driver for SQL Server installed as a system package (Windows: Control Panel > ODBC Data Sources to confirm; Linux/macOS: odbcinst -j).
Oracle	No extra driver install needed for the default thin-mode client.

5. Step 4: Configure `.env` for the business database

Open `.env` in a text editor and fill in the block under `--- Database connection ---`:

```
DB_TYPE=postgresql
DB_HOST=localhost
DB_PORT=5432
DB_NAME=your_database
DB_USER=your_readonly_user
DB_PASSWORD=your_password
```

`DB_TYPE` must be one of `postgresql`, `mysql`, `mssql`, or `oracle`. If your tables live under a specific schema rather than the connection's default one, also set `DB_SCHEMA`. If you'd rather supply a full SQLAlchemy connection string instead of the individual host/port/name fields, set `DB_CONNECTION_STRING` — when set, it takes priority and the individual `DB_*` fields above are ignored.

Connecting to more than one database is optional and not required for a first run. If you do want the app to auto-route each question to whichever of several databases looks relevant, [.env.example](#) documents the `DB_CONNECTIONS=name1, name2` plus `DB_<NAME>_*` pattern in detail — skip this for your first setup and add it later once the basic single-database setup is working.

6. Step 5: Verify the connection and build the schema index

Before starting the full application, confirm the database is reachable with exactly the credentials you just configured:

```
python scripts/test_db_connection.py
```

This prints a pass/fail result, the database version, and a table count — fix any reported error before continuing. Once it passes, build the schema index the AI uses to know what tables and columns exist (this reads your schema, not your data, and embeds it locally into a small on-disk index under [embeddings/.chroma/](#)):

```
python scripts/build_embeddings.py
```

Re-run this command any time your database's schema changes.

7. Step 6: Business-context knowledge retrieval (optional)

On top of the raw schema index above, the app can also retrieve business glossary terms, metric definitions, and curated example SQL queries to improve answer quality. This step is optional and the app works identically without it.

```
# Preview what would be ingested, without writing anything:
python -m scripts.ingest_schema --database-id default --dry-run

# Actually ingest (safe to re-run any time):
python -m scripts.ingest_schema --database-id default
```

Sample content lives under [data/knowledge/](#) — replace it with your own glossary/metrics/SQL examples before relying on this for anything beyond a demo; the shipped samples are clearly labeled placeholder content.

8. Step 7: User accounts and chat history (optional)

By default, this app has **no login screen** — anyone who can reach it can use it, the same as any other locally-run tool. If this new machine is genuinely single-user and only reachable by you, you can skip this entire step and go to Step 8.

If you want real sign-up/login accounts, with each person's chat history following them across browsers and devices, you need a **second, separate PostgreSQL database** — never the same database as your business data above, and never the same server login either.

Provision a local PostgreSQL server for this, however is easiest on this machine — install PostgreSQL directly, or run one in a container if you have Docker:

```
docker run --name text-to-sql-identity -e POSTGRES_PASSWORD=change_me -e POSTGRES_DB=text_to_sql
```

(If you already have PostgreSQL running locally for something else, just create a new, empty database on it instead — this feature does not need its own server, only its own database and its own schema.)

Then set these fields in `.env`:

```
LOCAL_AUTH_ENABLED=true
AUTH_DATABASE_URL=postgresql+psycopg2://postgres:change_me@localhost:5432/text_to_sql_identity
JWT_SECRET_KEY=
```

Generate a strong value for `JWT_SECRET_KEY` (this signs login sessions — never leave it blank when `LOCAL_AUTH_ENABLED=true`):

```
python -c "import secrets; print(secrets.token_urlsafe(48))"
```

Paste the printed value in as `JWT_SECRET_KEY=...`. Then apply this feature's database migrations — this creates all of the tables this feature needs inside the identity database (users, sessions, roles, conversations, and so on) and is safe to re-run any time you pull an update:

```
alembic -c identity/alembic.ini upgrade head
```

By default, sign-up happens through the dashboard's own "Sign up" link. If you'd rather provision the first account yourself instead of leaving open self-service sign-up on, leave `ALLOW_PUBLIC_REGISTRATION=false` (the default) and create the first administrator account with:

```
python scripts/bootstrap_admin.py
```

(This reads `BOOTSTRAP_ADMIN_ENABLED`, `BOOTSTRAP_ADMIN_EMAIL`, and `BOOTSTRAP_ADMIN_PASSWORD` from `.env` — set all three, run the script once, then it's safe to leave `BOOTSTRAP_ADMIN_ENABLED=false` afterward so a later restart never tries to re-provision it.)

Passwords must be at least 12 characters and clear a real strength check (no common patterns, nothing containing your email or name) — see [docs/authentication-and-password-policy.md](#) if a password is rejected and you're unsure why.

9. Step 8: Build the frontend

The React dashboard is the only UI this project ships, and it needs a one-time build step:

```
cd frontend
npm install
npm run build
cd ..
```

This produces `frontend/dist`, which the API process serves directly — you do not need to keep a separate frontend server running for normal use.

10. Step 9: Run the application

```
uvicorn api.main:app --host 0.0.0.0 --port 8000
```

Open <http://localhost:8000/> in a browser. The same process serves both the REST API and the built dashboard.

If you are actively changing frontend code rather than just using the app, run the API as above in one terminal, then in a second terminal:

```
cd frontend  
npm run dev
```

and open <http://localhost:5173/> instead — Vite's dev server proxies every API call through to the `uvicorn` process on port 8000, so no separate CORS configuration is needed either way.

11. Step 10: Verify it's working

Work through this checklist on the new machine before considering setup complete:

- `python scripts/test_db_connection.py` reports success against your business database.
- `http://localhost:8000/` loads the dashboard (or shows a sign-up/login screen, if you enabled Step 7).
- Typing a question and clicking through to "Confirm and Run" returns a real result table from your own database, not an error.
- If you enabled Step 7: signing up, then opening the same account in a different browser (or an incognito window) shows the same chat history — this confirms the identity database is actually being used rather than only browser-local storage.
- `docs/API.md` documents the full REST surface if you intend to call this app programmatically rather than through the dashboard.

12. Full .env reference for a fresh machine

The tables below cover only what's relevant to getting a new machine running — `.env.example` in the repository root documents every available setting, including advanced features (multi-source document/policy search, voice input, media generation/search) not needed for a first setup.

Required, business database:

Variable	Set it to
DB_TYPE	<code>postgresql</code> , <code>mysql</code> , <code>mssql</code> , or <code>oracle</code>
DB_HOST	Your database server's hostname or IP
DB_PORT	Your database server's port
DB_NAME	The database name to connect to
DB_USER	A dedicated read-only account
DB_PASSWORD	That account's password

Required, local LLM:

Variable	Set it to
OLLAMA_HOST	<code>http://localhost:11434</code> , unless Ollama runs elsewhere
OLLAMA_MODEL	<code>llama3.1:8b</code> , or another model you've pulled

Optional, user accounts and chat history (Step 7):

Variable	Set it to
LOCAL_AUTH_ENABLED	<code>true</code> to turn this feature on at all
AUTH_DATABASE_URL	A PostgreSQL connection string to a dedicated identity database
JWT_SECRET_KEY	A random secret — generate with the command shown in Step 7
ALLOW_PUBLIC_REGISTRATION	<code>true</code> for open sign-up, <code>false</code> to provision accounts yourself
BOOTSTRAP_ADMIN_ENABLED / _EMAIL / _PASSWORD	Set all three to create the first admin account, then turn the flag back off

Optional, deployment posture:

Variable	Set it to
ENVIRONMENT	<code>development</code> for a local machine; <code>production</code> requires an auth mechanism configured (<code>API_AUTH_TOKEN</code> , <code>OIDC_ISSUER</code> , or <code>LOCAL_AUTH_ENABLED</code>)
API_AUTH_TOKEN	A shared bearer secret, if you want a simple access check without full accounts

13. Alternative: running everything with Docker

If you'd rather not install Python/Node/Ollama directly on the new machine, this repository includes a Docker Compose setup that builds one image containing both the API and the built dashboard:

```
cp .env.example .env
```

Edit `.env` exactly as described in Steps 4–7 above, then:

```
docker compose build
docker compose up -d
docker compose exec api python scripts/build_embeddings.py
docker compose exec api python -m scripts.ingest_schema --database-id default
```

Ollama and your business database can either run on the host machine (reachable from inside the container) or as their own separate containers — see [docs/DEPLOYMENT.md](#) for external connectivity and reverse-proxy placement. The optional identity database from Step 7 can be provisioned the same way as shown there, as a normal PostgreSQL container, and referenced by `AUTH_DATABASE_URL` exactly as in the non-Docker path.

14. Common problems on a fresh machine

- `python scripts/test_db_connection.py` **fails with an authentication error.** Double-check `DB_USER/DB_PASSWORD` in `.env`, and confirm that account is actually allowed to connect from this machine's IP (a firewall or database-side allowlist is a common blocker on a new machine, not a credentials typo).
- **SQL Server: "driver not found" or similar.** Install the Microsoft ODBC Driver for SQL Server as a system package (not via `pip`) — see Step 1's prerequisites.
- `pip install -r requirements.txt` **fails building a package from source.** This most often means the machine's Python version is newer than this project's pinned dependencies expect. Install Python 3.11 specifically and recreate the virtual environment against it.
- **The dashboard loads but questions never return results.** Confirm Ollama is actually running (`ollama serve`, or check it's running as a service) and that `OLLAMA_HOST` in `.env` matches where it's listening.
- **Chat history doesn't follow you to a different browser.** This only works if Step 7 (the identity database) was configured — with `LOCAL_AUTH_ENABLED=false` (the default), history is deliberately local to that one browser only.
- **Forgot which account is the administrator.** Re-run `python scripts/bootstrap_admin.py` after setting `BOOTSTRAP_ADMIN_ENABLED=true` again — it is idempotent and safe to re-run; it will not create a duplicate if the account already exists.

15. Security checklist before relying on this outside your own laptop

- The business database account in `.env` is genuinely read-only at the database level, not just relying on this app's own SQL validation.
- `.env` itself is never committed to git and is not readable by anyone besides you on this machine.
- If more than one person will use this machine, or it's reachable over a network rather than only `localhost`, turn on an authentication mechanism (`LOCAL_AUTH_ENABLED=true`, `API_AUTH_TOKEN`, or `OIDC_ISSUER`) rather than leaving the app open with no login at all.
- If you expose this app beyond your own machine, read `SECURITY.md` and `docs/DEPLOYMENT.md` first — this guide only covers a local, single-machine setup.